

Communication Complexity: Communication Lower Bounds For Multi-Party Graph Traversal

Ryan Batubara
rbatubara@ucsd.edu

Ivy Hawks
m1hawks@ucsd.edu

Darren Ho
rbatabura@ucsd.edu

Ciro Zhang
ciz001@ucsd.edu

Dr. Shachar Lovett
shachar.lovett@gmail.com

Communication Complexity: Communication Lower Bounds For Multi-Party Graph Traversal

Ryan Batubara

rbatubara@ucsd.edu

Ivy Hawks

m1hawks@ucsd.edu

Darren Ho

rbatubara@ucsd.edu

Ciro Zhang

ciz001@ucsd.edu

Dr. Shachar Lovett

shachar.lovett@gmail.com

Motivation

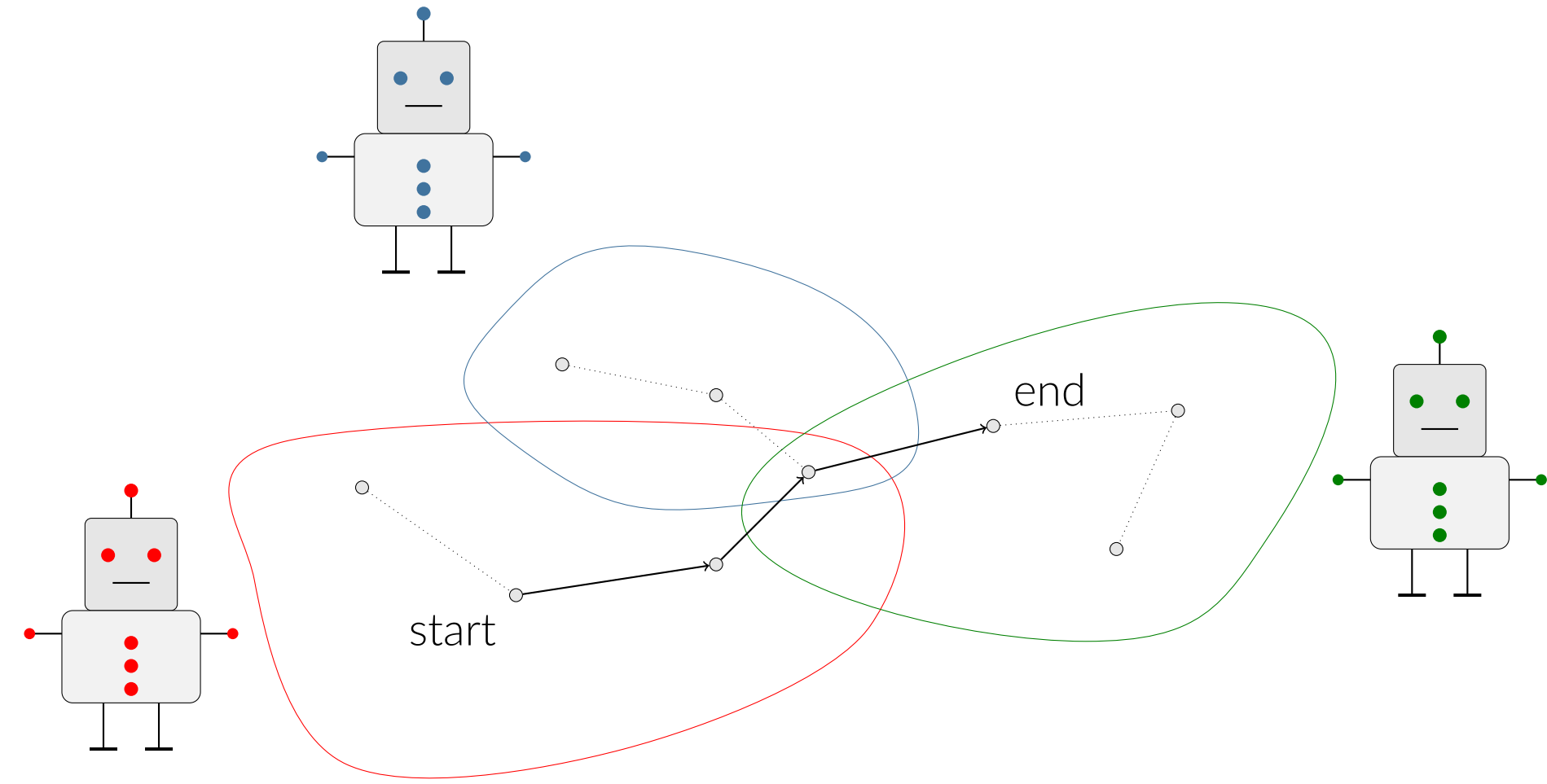


Figure 1. Path in Agent Owned Graphs

In multi-agentic learning systems, there is a strong need for multiple agents to be able to coordinate while learning a problem. However, it is also important for those agents to be able to communicate the minimum amount of information possible, so as to avoid one agent dominating the training process. It is therefore natural to consider questions in Communication complexity, which asks, for any function,

How much communication is fundamentally necessary?

In our model, we consider the learning space as a graph, which each agent will need to understand properties of in order to find ideal training parameters.

Two Party Communication Complexity

Communication Complexity studies the minimum number of bits that multiple parties must exchange to compute a function when the input is distributed.

Instead of asking: *How fast is the algorithm?*

We ask: *How much communication is needed within the algorithm?*

Two Party Communication Complexity

There are two players, traditionally called **Alice** and **Bob**.

- Alice holds input x
- Bob holds input y

Neither of them knows the inputs of the other persons, so they must talk back and forth until they know enough to compute some function $f(x, y)$

In the worst case, Alice just send all n bits. This is called **maximally hard**, as after sending her entire input, Bob can simply compute the result.

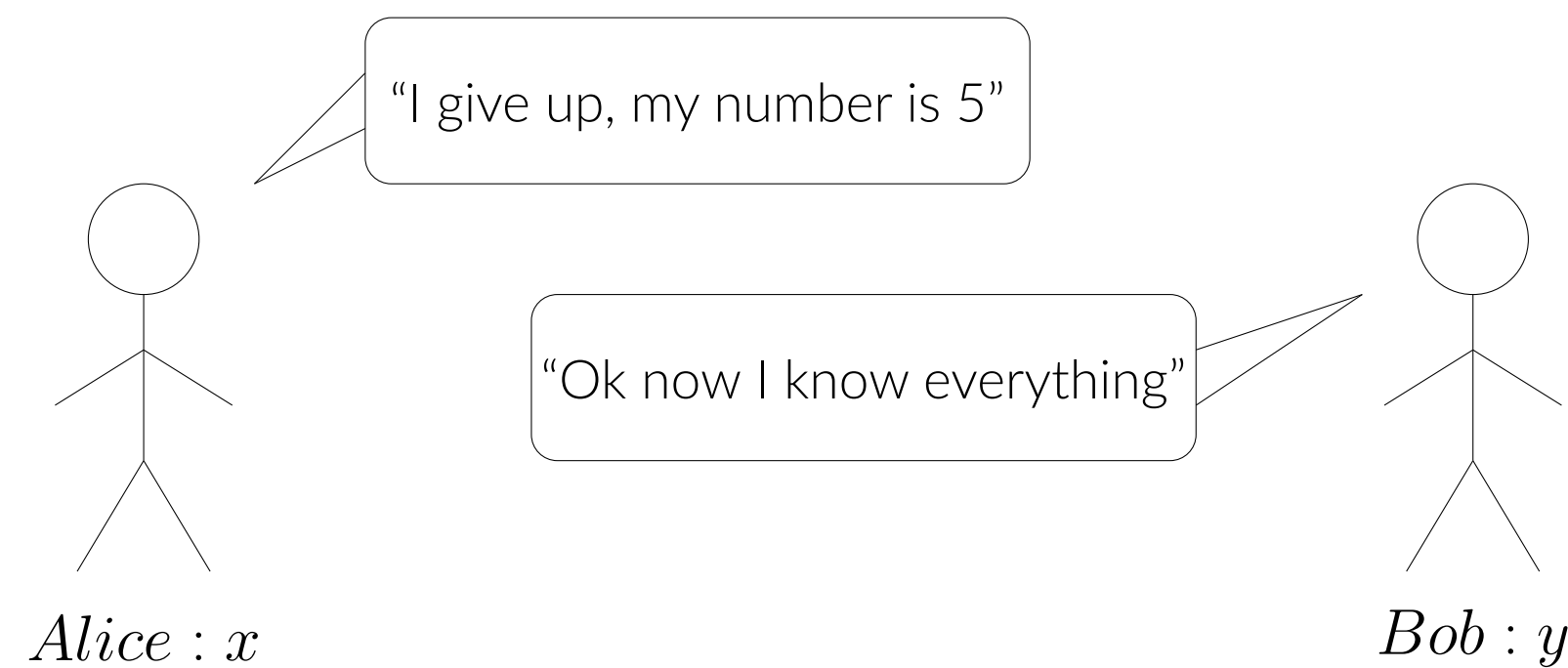


Figure 2. Alice and Bob in the Maximally Hard case

Equality is Maximally Hard

Equality Problem: Do Alice and Bob have the same n -bit string?

$$EQ(x, y) = \begin{cases} 1 & x = y \\ 0 & x \neq y \end{cases}$$

In figure 3, we represent EQ as a communication matrix:

- Rows = Alice's input x
- Columns = Bob's input y
- Blue dots mark where $EQ(x, y) = 1$

In communication complexity, a correct protocol must divide this matrix into rectangles that contain either all 0's or all 1's.

But each blue dot is isolated, so in order to cover all 2^n diagonal 1's, the protocol needs at least 2^n rectangles. It turns out this implies that Equality requires at least n bits of communication.

Thus, **Equality is maximally hard**.

The Traversal Problem

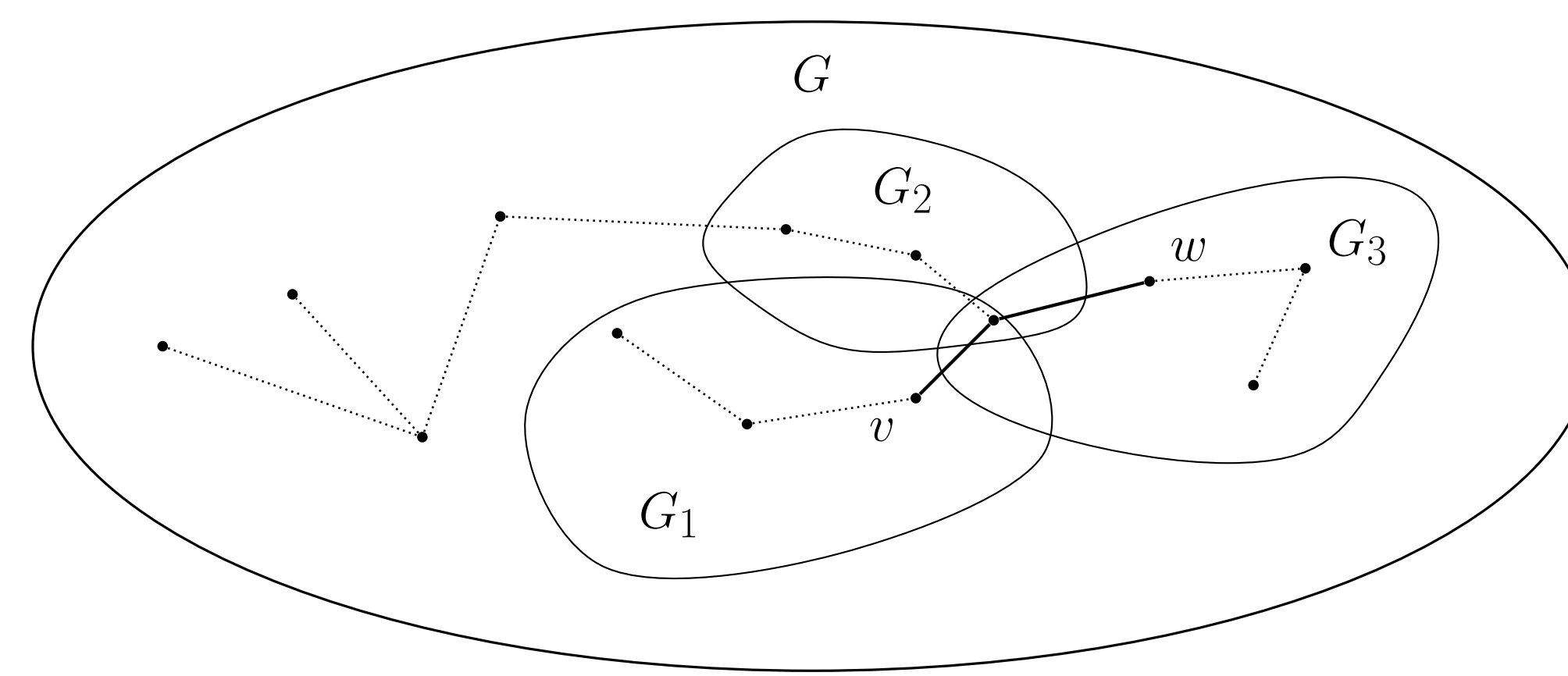


Figure 4. Traversal example where a valid path from v to w is shown in bold.

In this section, we define a simpler version of the problem from our motivation.

Consider a large graph G . Inside G , each agent "owns" part of the graph. We call these subgraphs $G_1, G_2, \dots, G_k$. No single player sees the entire graph.

Given a starting vertex v and a target vertex w , how much information do the agents need to share about their subgraphs to determine whether there is a path from v to w ?

For this poster, we will only consider a two agent case.

Embedding Equality into Traversal Equality Gadget Chain

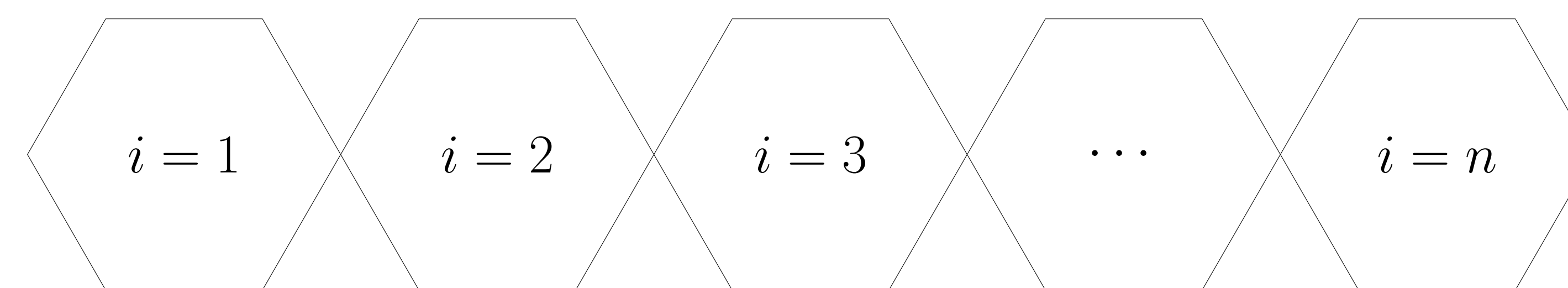


Figure 6. Chain of Equality Gadgets

To prove that Traversal, like Equality, is also maximally hard we show that we can embed Equality in Traversal.

By chaining the gadgets together, we turn each bit comparison into a small "gate." The chain is traversable only if every gate opens, which happens exactly when $x_i = y_i$ for all i , in other words, when $x = y$.

Therefore, any protocol that solves Traversal can also solve Equality. Since Equality is deterministically maximally hard, **Traversal is also deterministically maximally hard**.

	Bob: y				
	00	01	10	11	
Alice: x	00	●	●	●	● $f(x, y) = 1$ ● $f(x, y) = 0$
	01	●	●	●	
	10	●	●	●	
	11	●	●	●	

Figure 3. Communication matrix for Equality

The Equality Gadget

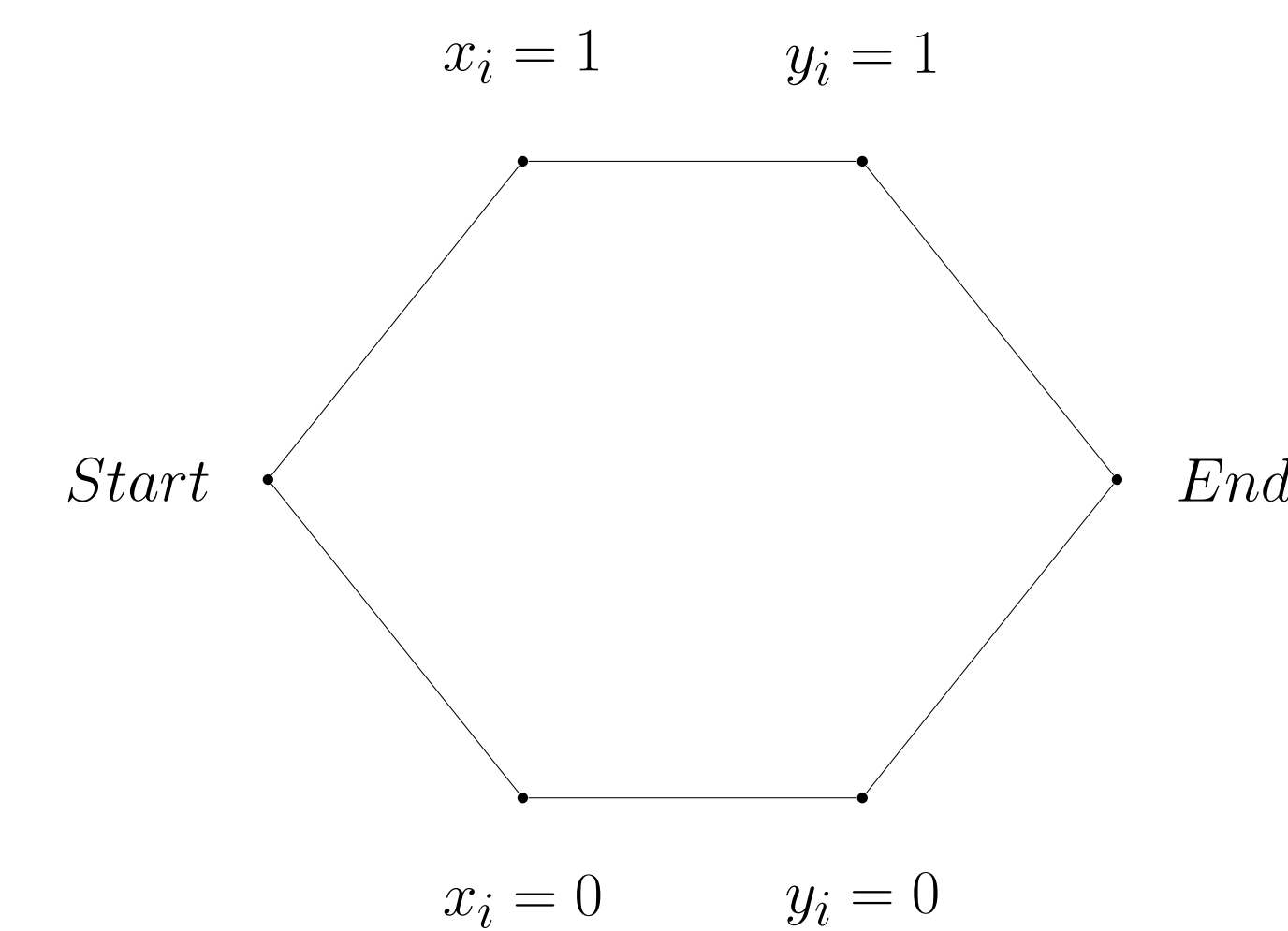


Figure 5. Gadget for proving equality Embedding

- Top branch corresponds to $x_i = y_i = 1$,
- Bot branch corresponds to $x_i = y_i = 0$.

Consider this gadget where its only Traversable from Start to End if both x_i and y_i are both 0 or 1. In other words when $y_i = x_i$

Introducing Randomness

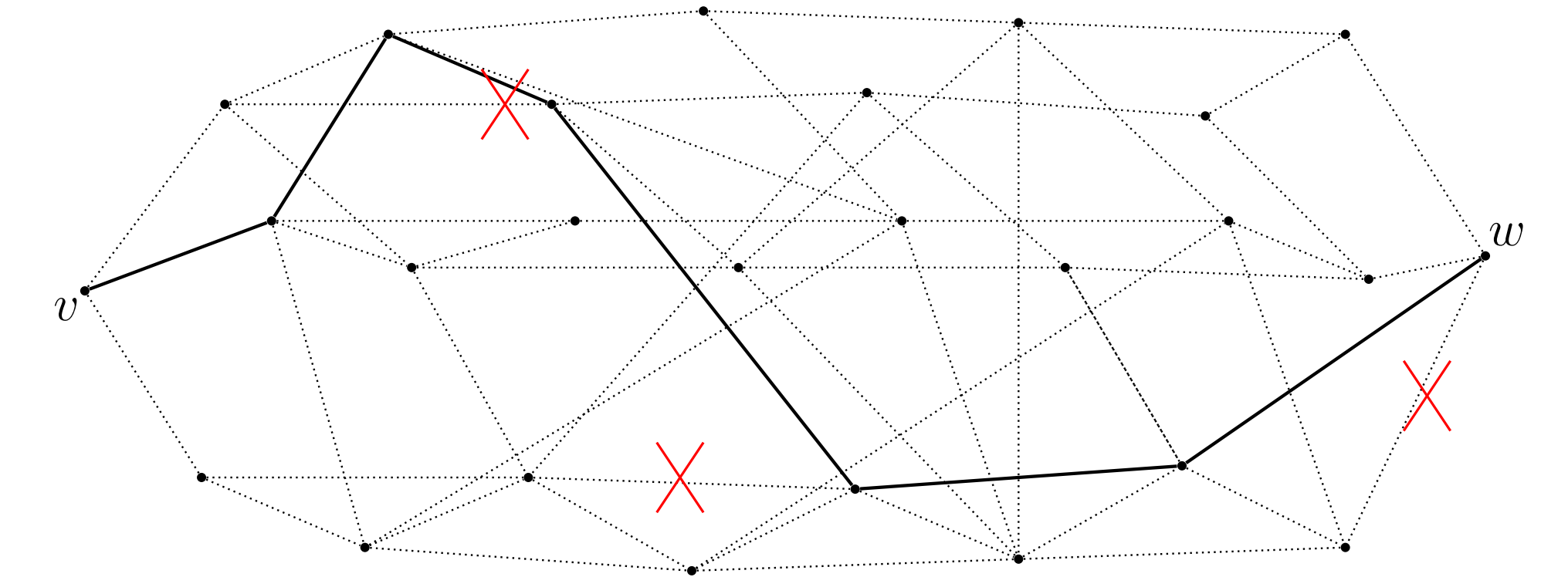


Figure 7. Graph with Skipped Edges

Deterministically we have seen that this is **maximally hard**. There is no better way to solve this problem than for Alice to send her entire input to Bob. However, we might also consider adding randomness, and allowing for error. For instance, if there is a path, it is unlikely that any randomly selected vertex will fall on it, so we might consider randomly removing some.

Set Disjointness

To find a lower bound for a random algorithm we can no longer rely on embedding equality, as random algorithm's for equality can be bounded as low as $\Theta(1)$. Thus, we turn to set disjointness.

$$DISJ(x, y) = \begin{cases} 0 & x \cap y \neq \emptyset \\ 1 & x \cap y = \emptyset \end{cases}$$

Set Disjointness gives each player some number of elements from a set, and asks if the players have any elements in common. If so, the protocol outputs 0, and 1 otherwise. Set Disjointness is usefull when studying random algorithms since it has been shown that it requires **maximal random communication**.

Disjoint Gadget and Embedding

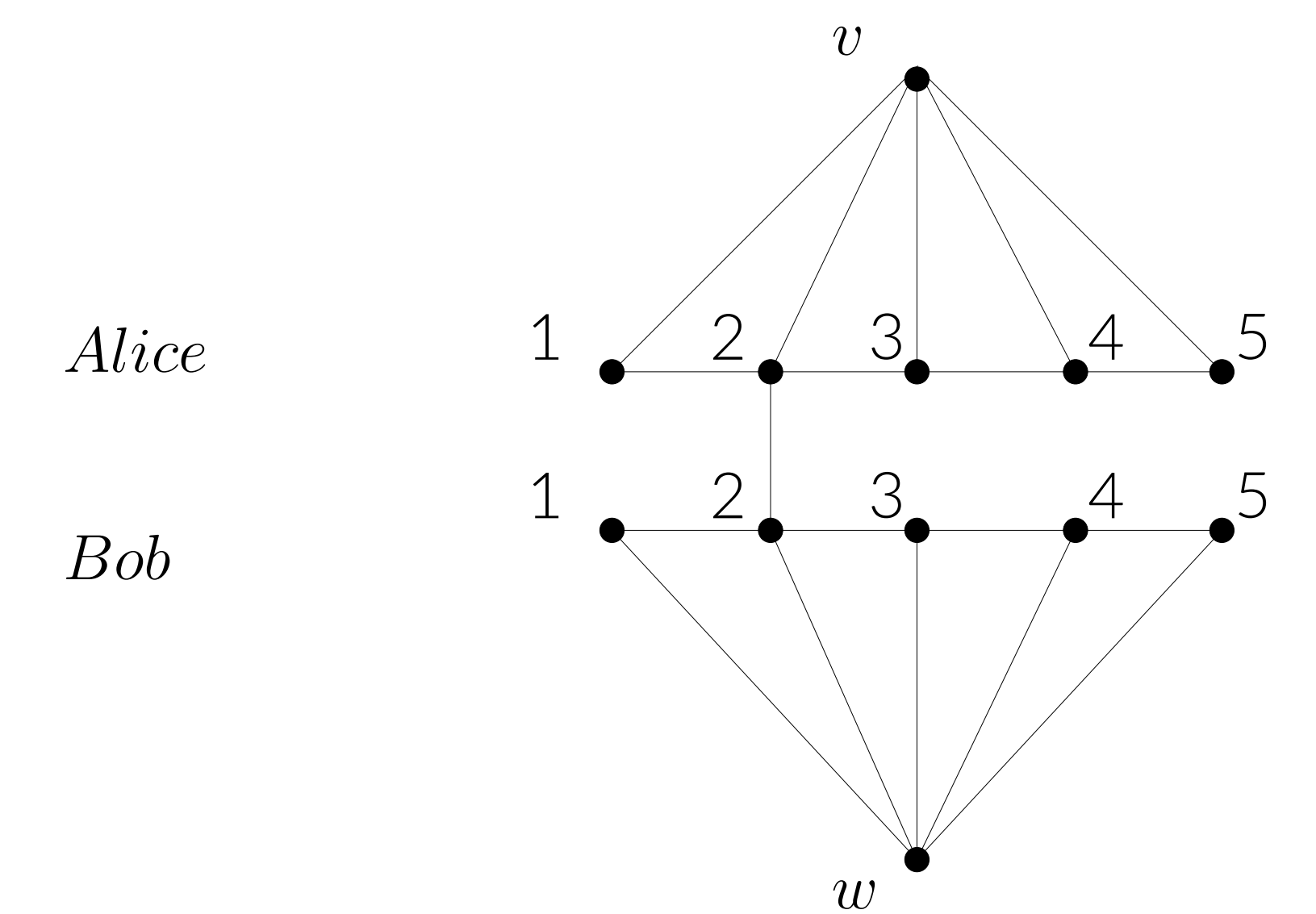


Figure 8. Disjointness Gadget. Here, a path exists as Alice and Bob both share element 2

This gadget can also be embedded into Traversal. Here, edges between Alice's side and Bob's side only exist if both players have an element in common